

ALGORITHME SystemeCompleetNLP_Malagasy

// Initialisation

CHARGER dictionnaire_malgache

CHARGER regles_linguistiques

CHARGER corpus_entrainement

INITIALISER cache_resultats

// Boucle principale

TANT QUE vrai FAIRE

// Attente de requête utilisateur

requete = ATTENDRE_REQUETE()

// Traitement asynchrone

EN PARALLÈLE:

// Analyse linguistique

analyse = analyser_texte(requete.texte)

// Suggestions de correction

corrections = suggerer_corrections(requete.texte)

// Autocomplétion

suggestions = generer_suggestions(requete.texte, requete.contexte)

// Traduction

traductions = traduire_mots_cles(requete.texte)

FIN PARALLÈLE

// Fusion des résultats

resultat = fusionner_resultats(analyse, corrections, suggestions, traductions)

```
// Ajout de métadonnées
resultat.confidence = calculer_confiance_totale(resultat)
resultat.temps_reponse = mesurer_temps()

// Envoi au client
ENVOYER_REPONSE(resultat)

// Apprentissage continu (si activé)
SI mode_apprentissage ACTIF ALORS
    mettre_a_jour_modeles(requete, resultat)
FIN SI
FIN TANT QUE
FIN ALGORITHME

ALGORITHME analyser_texte(texte)
    resultat = {}

    // 1. Analyse lexicale
    resultat.mots = tokeniser(texte)
    resultat.lemmes = lemmatiser(texte)
    resultat.categories = etiqueter_parties_discours(texte)

    // 2. Analyse syntaxique (basique)
    resultat.structure = analyser_structure_phrase(texte)

    // 3. Analyse sémantique (basique)
    resultat.sentiment = analyser_sentiment(texte)
    resultat.themes = extraire_themes(texte)
```

```
// 4. Détection d'entités
```

```
resultat.entites = reconnaitre_entites_nommees(texte)
```

```
RETOURNER resultat
```

```
FIN ALGORITHME
```

```
ALGORITHME fusionner_resultats(...resultats)
```

```
fusion = {}
```

```
// Fusion par pondération de confiance
```

```
POUR CHAQUE source DANS resultats FAIRE
```

```
    POUR CHAQUE cle, valeur DANS source FAIRE
```

```
        SI cle  $\notin$  fusion ALORS
```

```
            fusion[cle] = []
```

```
        FIN SI
```

```
        fusion[cle].append({
```

```
            'valeur': valeur,
```

```
            'source': source.nom,
```

```
            'confidence': source.confidence
```

```
        })
```

```
    FIN POUR
```

```
FIN POUR
```

```
// Résolution pour chaque clé
```

```
resultat_final = {}
```

```
POUR CHAQUE cle DANS fusion FAIRE
```

```
    SI fusion[cle].length == 1 ALORS
```

```
        // Une seule source
```

```
    resultat_final[cle] = fusion[cle][0].valeur  
SINON  
    // Multiple sources, prendre la meilleure  
    meilleure = selectionner_meilleure_source(fusion[cle])  
    resultat_final[cle] = meilleure.valeur  
    resultat_final[cle + '_sources'] = fusion[cle]  
FIN SI  
FIN POUR  
  
RETOURNER resultat_final  
FIN ALGORITHME
```